

Salish Sea Creatives — Motion Upgrade BDD Spec

Project: salishseacreative.com motion/interaction upgrade **Stack baseline:** Astro (static, no framework islands), Bricolage Grotesque + DM Sans + JetBrains Mono, single CSS bundle in `/_astro/`, no JS dependencies today **Reference implementation:** lcm.design (Locomotive Scroll smooth scroll + parallax, GSAP hover-reveal, Swiper carousels, fx CSS reveals) **Scope of this spec:** Phase A (Lenis smooth scroll, scroll reveals, Portage text drift) + Phase B (GSAP ScrollTrigger parallax, cursor-follow hover reveal) **Out of scope (future Phase C):** Swiper galleries on audience pages, magnetic buttons, Astro View Transitions, page loader

1. Global constraints

These apply to every feature below. They are non-negotiable acceptance criteria.

#	Constraint	Value
G1	Desktop gating	Motion features initialize only at viewport min-width: 1024px (matchMedia), mirroring LCM' s 790px/1090px gates
G2	Reduced motion	All motion disabled when prefers-reduced-motion: reduce
G3	No jQuery	Vanilla JS + Lenis + GSAP 3 only
G4	No-JS fallback	Site fully readable with JS disabled; all content visible without animation (reveals must not hide content permanently)
G5	Performance budget	Added JS 60 KB gzipped total (lenis ~7 KB + gsap core + ScrollTrigger ~40 KB); Lighthouse performance 95 on / after upgrade
G6	Astro-native	Scripts ship as Astro bundled scripts (<code><script></code> in components/ layout) —no CDN script tags, npm-installed packages only (lenis, gsap)
G7	Cleanup	All rAF loops, ScrollTriggers, and listeners torn down on astro:before-swap / page unload to survive future View Transitions

2. Feature files

Feature 1 — Smooth inertial scrolling

Feature: Smooth inertial scrolling

As a visitor on desktop
I want the page to scroll with smooth inertia
So that the site feels crafted and intentional

Background:

Given the Lenis library is installed and initialized on the root layout
And Lenis is configured with lerp-based smoothing equivalent to inertia **0.7**

@desktop @motion

Scenario: Lenis initializes on desktop with motion allowed

Given I visit "/" with a viewport of **1440x900**
And my OS does not request reduced motion
When the page finishes loading
Then Lenis is attached to the document scroller
And the html element carries a "**lenis**" class

@desktop @motion

Scenario: Anchor navigation still works

Given I am on "/" with Lenis active
When I click the nav link to "**#templates**"
Then the page smooth-scrolls to the templates section
And the URL hash updates to "**#templates**"

@mobile

Scenario: Lenis does not initialize on mobile

Given I visit "/" with a viewport of **390x844**
When the page finishes loading
Then Lenis is not initialized
And native browser scrolling is used

@a11y

Scenario: Reduced motion disables Lenis

Given I visit "/" with "**prefers-reduced-motion: reduce**"
Then Lenis is not initialized
And native browser scrolling is used

Implementation notes

- Init inside a single `MotionManager` client script in the root layout; gate on `matchMedia('(min-width: 1024px)')` AND `matchMedia('(prefers-reduced-motion: no-preference)')`.
- Wire Lenis `scroll` event to `ScrollTrigger.update` (Feature 4 depends on this).
- Handle in-page anchors via `lenis.scrollTo()` so `#templates` / `#migration` nav links animate.

Feature 2 — Scroll-triggered section reveals

```
Feature: Scroll-triggered content reveals
  As a visitor scrolling the page
  I want content blocks to fade and slide in as they enter the viewport
  So that the page feels alive instead of static

Background:
  Given reveal-eligible elements carry the class ".fx"
  And a shared IntersectionObserver manages reveal state

@desktop @motion
Scenario: Elements reveal on entering viewport
  Given I am on "/" with motion enabled
  And a ".fx" element is below the viewport
  When I scroll until the element is 15% visible
  Then the element receives the "is-visible" class within 200ms
  And it transitions opacity from 0 to 1
  And it translates no more than 24px vertically

@desktop @motion
Scenario: Reveals fire once and stay
  Given a ".fx" element has been revealed
  When I scroll it out of and back into the viewport
  Then it remains fully visible

@a11y @no-js
Scenario: Content is never hidden without motion
  Given I visit "/" with reduced motion OR with JavaScript disabled
  Then every ".fx" element is fully visible immediately
  And no element has opacity 0 in computed style
```

Implementation notes

- Hide via `.js .fx { opacity: 0 }` — the `js` class is already toggled by the existing inline script (`documentElement.classList.add('js')`), which satisfies G4 for free. Reduced-motion users get `.fx` visible via a `prefers-reduced-motion` CSS override; the observer must also not run for them (double safety).
- Transition timing: 600–800ms ease-out, stagger siblings by 60ms via `transition-delay` set from element index. Slow and intentional — LCM pacing, not SaaS-snappy.
- Apply `.fx` to: hero subline, the three audience cards, Portage section blocks, studio values section, footer CTA.

Feature 3 — Portage horizontal text drift

```
Feature: Portage migration list horizontal drift
  As a visitor scrolling past the Portage section
  I want the migration-path rows to slide horizontally in opposing directions
  So that the "ferry" metaphor reads as motion across the page
```

Background:

Given the Portage section ("**#migration**") contains rows of migration pairs
(ghost2astro, squarespace2astro, substack2astro, webflow2astro)
And rows alternate data-drift-direction between "**left**" and "**right**"

@desktop @motion

Scenario: Rows translate horizontally with scroll progress

Given I am on "/" with motion enabled at 1440x900
When I scroll through the Portage section
Then odd rows translate toward the left proportional to scroll progress
And even rows translate toward the right proportional to scroll progress
And total travel does not exceed 15% of row width in either direction

@all

Scenario: Drift is purely visual

Given motion is enabled
Then all migration pair links remain clickable at all times
And text remains selectable
And tab order is unchanged

Implementation notes

- Drive from the same IntersectionObserver + rAF scroll handler (or ScrollTrigger if Feature 4 ships in the same PR — prefer one mechanism per PR).
- Pure transform: translateX(); never animate left/right.
- This replaces the current static ghost2astro ... row. Content stays as real <a> elements.

Feature 4 — Parallax template lane cards (GSAP ScrollTrigger)

Feature: Parallax on template lane cards

As a visitor scrolling the templates section
I want the three lane cards to drift at different vertical speeds
So that the section has depth, like the reference site's hero images

Background:

Given GSAP 3 and ScrollTrigger are npm-installed and bundled
And the templates section ("**#templates**") contains three lane cards
And the cards carry data-speed attributes of 3, -1, and 2 respectively

@desktop @motion

Scenario: Cards parallax at distinct speeds

Given I am on "/" with motion enabled at 1440x900
When I scroll the templates section through the viewport
Then each card's translateY offset differs proportionally to its data-speed
And the card with data-speed="-1" moves opposite to scroll direction

@desktop @motion

Scenario: Parallax syncs with Lenis

```
Given Lenis is active (Feature 1)
Then ScrollTrigger updates are driven by the Lenis scroll event
And no scroll jitter or double-transform is visible
```

@mobile @a11y

Scenario: No parallax on mobile or reduced motion

```
Given I visit "/" at 390x844 OR with reduced motion
Then ScrollTrigger is not created for the lane cards
And all cards have no transform applied
```

Implementation notes

- Map data-speed to a y offset via ScrollTrigger scrub: true; cap travel at ± 60 px.
- will-change: transform on cards during active scroll only (add/remove class) to avoid permanent layer cost.
- If Phase A ships alone first, implement this with a plain rAF + scroll handler and swap to ScrollTrigger in Phase B — do not ship both mechanisms.

Feature 5 — Cursor-follow hover reveal on lane cards

Feature: Cursor-follow template preview

As a visitor hovering a template lane
I want a floating template screenshot to follow my cursor
So that I get a visual preview before committing to a click

Background:

```
Given each lane card has a data-preview-img attribute pointing to an optimized WebP screenshot
And the reveal element is appended on first hover, not on page load
```

@desktop @motion

Scenario: Preview appears and follows cursor

```
Given I am on "/" with motion enabled, pointer: fine
When I hover a lane card
Then a floating preview element fades in within 300ms
And the preview follows the cursor with eased lag (GSAP quickTo or equivalent)
And the preview image is the card's data-preview-img
```

@desktop @motion

Scenario: Preview dismisses cleanly

```
Given a preview is visible
When my cursor leaves the lane card
Then the preview fades out and is removed from the DOM
```

@desktop @motion

Scenario: Preview never traps navigation

```
Given a preview is visible
When I click the lane card
Then navigation to the lane page occurs normally
```

```

@mobile @touch
Scenario: No hover preview on touch devices
  Given I visit "/" with a coarse pointer
  Then no preview element is created on tap
  And tapping a lane card navigates directly

@a11y
Scenario: Keyboard users get an equivalent affordance
  Given I navigate by keyboard
  When a lane card receives focus
  Then the preview appears anchored near the card
  And it is hidden from the accessibility tree (aria-hidden)

```

Implementation notes

- Direct port of the Codrops HoverImgFx4 pattern (the same code lcm.design ships in `reveal-img.js`), modernized: GSAP 3 `quickTo` instead of `TweenMax`, no jQuery, `pointer: fine` media query gate.
- Preview screenshots: 480px-wide WebP, `loading="lazy"`, generated from template demo screenshots. Placeholder images acceptable for first release.
- `pointer-events: none` on the preview element — non-negotiable, or it will eat clicks.

Feature 6 — Global accessibility and fallback behavior

```

Feature: Accessibility and resilience of the motion system

@a11y
Scenario: Full reduced-motion pass
  Given I visit any page with "prefers-reduced-motion: reduce"
  Then Lenis, ScrollTrigger, drift, and hover previews are all inert
  And all content is visible and readable
  And scrolling is native

@no-js
Scenario: No-JS pass
  Given JavaScript is disabled
  Then every page renders all content visibly
  And all links and navigation work

@a11y
Scenario: Motion does not break focus visibility
  Given motion is enabled
  When I tab through the page
  Then focus outlines are visible on every interactive element
  And parallax/drift transforms do not clip focus rings

```

3. Playwright test matrix

Map each feature to one spec file under `tests/motion/`. Run against the Astro preview server (`npm run preview`).

Spec file	Scenarios covered	Key Playwright techniques
<code>lenis.spec.ts</code>	F1 all	<code>expect(page.locator('html')).toHaveClass('lenis');</code> ; hash assertion after nav click; viewport 390px negative case
<code>reveals.spec.ts</code>	F2	Scroll via <code>mouse.wheel(0, delta);</code> ; assert <code>is-visible</code> class and <code>getComputedStyle opacity</code> ; reload with <code>reducedMotion: 'reduce'</code> context and assert opacity 1 immediately
<code>drift.spec.ts</code>	F3	Capture <code>boundingBox()</code> of a drift row at two scroll positions; assert <code>translateX</code> delta sign per row parity
<code>parallax.spec.ts</code>	F4	<code>page.evaluate</code> to read <code>transform</code> on each card at two scroll positions; assert distinct offsets per data-speed; negative case at mobile viewport
<code>hover-reveal.spec.ts</code>	F5	<code>page.hover()</code> on lane card; assert preview element appears/moves with <code>mouse.move()</code> ; <code>pointerEvents: 'none'</code> computed check; touch emulation via <code>hasTouch: true</code> context
<code>ally.spec.ts</code>	F6 + G4	<code>reducedMotion: 'reduce'</code> context full pass; keyboard Tab walk with <code>:focus-visible</code> assertion; axe-core scan (<code>@axe-core/playwright</code>) on /

Context matrix — every spec runs under three contexts:

```
const contexts = [  
  { name: 'desktop-motion', viewport: { width: 1440, height: 900 } },  
  { name: 'desktop-reduced', viewport: { width: 1440, height: 900 }, reducedMotion: 'reduce' },  
  { name: 'mobile', viewport: { width: 390, height: 844 }, hasTouch: true },  
];
```

Motion assertion helper — don't assert exact pixel values (Lenis lerp is timing-dependent). Assert directional deltas:

```
async function transformOf(locator: Locator) {
```

```
return locator.evaluate(el => new DOMMatrixReadOnly(getComputedStyle(el).transform));
}
// Then: expect(m2.m42).toBeGreaterThan(m1.m42) etc.
```

4. Definition of done (PR-level)

- All Gherkin scenarios have corresponding Playwright tests; suite green in CI
- G5 budget verified: astro build output shows motion JS chunk 60 KB gzipped (npx bundle size or manual check on dist/_astro/*.js)
- Lighthouse CI: performance 95, accessibility = 100 on /
- axe-core: zero violations on /
- Manual QA: Safari, Firefox, Chrome at 1440px; iOS Safari at 390px
- Feature flags: each phase ships behind its own commit; Phase A must be shippable and revertible independently of Phase B

5. Handoff notes for the coding agent

1. Create src/scripts/motion.ts as the single entry point — Lenis init, observer setup, GSAP registration, all gated by the two matchMedia checks. Import it once from the root layout.
2. Do not edit the existing inline no-js → js class script; build on it.
3. The reference behavior to match is documented at lcm.design — its data-scroll-* attribute semantics are the target feel, but implement with Lenis + GSAP 3, not Locomotive Scroll.
4. The hover-reveal reference is the Codrops “Image Reveal Hover Effects” demo, effect variant 4 (HoverImgFx4).
5. Respect G7 cleanup from day one — View Transitions are coming in Phase C.